

Linear algebra for this course

Vectors, distances, and dissimilarity matrices, with the numbers written out

Fall 2026 · read at your own pace; every symbol here is the one the lectures use

Who this is for

Several of you said that the linear algebra in the first lectures went by quickly. This handout slows it down. It contains nothing that is not already in the lectures; it just takes each object one at a time, defines it, and works a small numerical example by hand so that you can see what the symbols are doing. Vectors here have two or three entries so the arithmetic fits on a line. The vectors in the course have thousands of entries, and nothing changes except the length of the sums.

The one idea. A list of D numbers is a point in a D -dimensional space. Once two things are points, “how similar are they” becomes “how far apart are they”, and there is more than one reasonable answer to that. Everything below is about making that sentence precise.

1 Vectors, and the names the course uses

1.1 A vector is a list of numbers, and also a point

A *vector* is an ordered list of numbers. Write it in bold, $\mathbf{x}$, and write its j -th entry as x_j :

$$\mathbf{x} = (x_1, x_2, \dots, x_D).$$

D is the number of entries, the *dimension*. Two readings of the same list are worth holding at once. As a **point**, $\mathbf{x}$ is a location: with $D = 2$, $(3, 4)$ is the point three units right and four units up. As an **arrow**, $\mathbf{x}$ is the displacement from the origin to that point, with a length and a direction. The lectures use both.

Example. Record the firing rates of three neurons to one image and get $\mathbf{x} = (2, 0, 5)$: neuron 1 fired at 2 spikes per second, neuron 2 was silent, neuron 3 fired at 5. That image is now a point in a three-dimensional “neuron space”, one axis per neuron. Pixel values, a row of network activations, and a set of ratings are all vectors in exactly this sense.

1.2 Stimuli are indexed with a superscript in parentheses

The course looks at many stimuli at once, so it needs two indices: *which stimulus* and *which entry*. The convention, used in every lecture and assignment:

- $\mathbf{x}^{(i)}$ is the response vector to stimulus i , for $i = 1, \dots, N$;
- $x_j^{(i)}$ is its j -th entry, for $j = 1, \dots, D$.

So N counts stimuli (images, trials, conditions) and D counts entries (pixels, units, neurons, rating scales). Read $x_3^{(7)}$ as “the response of neuron 3 to image 7”.

1.3 Stacking the vectors: the response matrix

Put the N response vectors on top of each other, one per row, and you get the *response matrix* $\mathbf{X}$, with N rows and D columns:

$$\mathbf{X} = \begin{pmatrix} - & \mathbf{x}^{(1)} & - \\ - & \mathbf{x}^{(2)} & - \\ & \vdots & \\ - & \mathbf{x}^{(N)} & - \end{pmatrix} \quad \mathbf{X} \text{ is } N \times D, \quad (\mathbf{X})_{ij} = x_j^{(i)}.$$

Example. Three images, two neurons ($N = 3$, $D = 2$), with $\mathbf{x}^{(1)} = (3, 4)$, $\mathbf{x}^{(2)} = (4, 3)$, $\mathbf{x}^{(3)} = (0, 1)$:

$$\mathbf{X} = \begin{pmatrix} 3 & 4 \\ 4 & 3 \\ 0 & 1 \end{pmatrix}, \quad \text{row 2 is } \mathbf{x}^{(2)}, \quad \text{column 2 is neuron 2's response to every image.}$$

“ $N \times D$ ” always means rows first, then columns. **Rows are stimuli; columns are features.** Every data matrix in this course is laid out this way.

In NumPy: $\mathbf{X}.\text{shape}$ is (N, D) ; $\mathbf{X}[i]$ is row i ; $\mathbf{X}[:, j]$ is column j .

2 Adding, subtracting, and scaling vectors

Vectors of the same dimension are added and subtracted entry by entry, and multiplied by a number (a *scalar*) entry by entry:

$$\mathbf{x} + \mathbf{y} = (x_1 + y_1, \dots, x_D + y_D), \quad \mathbf{x} - \mathbf{y} = (x_1 - y_1, \dots, x_D - y_D), \quad c\mathbf{x} = (cx_1, \dots, cx_D).$$

Example. With $\mathbf{x} = (3, 4)$ and $\mathbf{y} = (4, 3)$:

$$\mathbf{x} + \mathbf{y} = (7, 7), \quad \mathbf{x} - \mathbf{y} = (-1, 1), \quad 2\mathbf{x} = (6, 8), \quad \frac{1}{2}\mathbf{x} = (1.5, 2).$$

Geometrically, $\mathbf{x} + \mathbf{y}$ is the arrow you get by placing $\mathbf{y}$'s tail at $\mathbf{x}$'s head; $c\mathbf{x}$ points the same way as $\mathbf{x}$ (the opposite way if $c < 0$) and is $|c|$ times as long.

The difference vector. $\mathbf{x} - \mathbf{y}$ is the arrow *from* the point $\mathbf{y}$ to the point $\mathbf{x}$. Its length is the distance between them. That one sentence is the whole of §4, so it is worth a second read: subtracting two response vectors gives the displacement between two stimuli in neuron space.

In NumPy: $\mathbf{x} + \mathbf{y}$, $\mathbf{x} - \mathbf{y}$, $2 * \mathbf{x}$, all entry by entry.

3 The dot product

3.1 Definition

The *dot product* of two vectors of the same dimension multiplies matching entries and adds the results. It turns two vectors into one number:

$$\mathbf{x} \cdot \mathbf{y} = \sum_{j=1}^D x_j y_j = x_1 y_1 + x_2 y_2 + \cdots + x_D y_D.$$

Example. $(3, 4) \cdot (4, 3) = 3 \cdot 4 + 4 \cdot 3 = 12 + 12 = 24$. $(3, 4) \cdot (0, 1) = 0 + 4 = 4$. $(3, 4) \cdot (4, -3) = 12 - 12 = 0$.

The dot product is symmetric, $\mathbf{x} \cdot \mathbf{y} = \mathbf{y} \cdot \mathbf{x}$, and a vector dotted with itself gives the sum of its squared entries: $(3, 4) \cdot (3, 4) = 9 + 16 = 25$.

3.2 The Euclidean norm, or length

The *norm* of $\mathbf{x}$ is its length as an arrow, by Pythagoras:

$$\|\mathbf{x}\| = \sqrt{\mathbf{x} \cdot \mathbf{x}} = \sqrt{\sum_j x_j^2}.$$

Example. $\|(3, 4)\| = \sqrt{9 + 16} = 5$. $\|(4, 3)\| = 5$. $\|(0, 1)\| = 1$. $\|(1, 1, 1)\| = \sqrt{3} \approx 1.732$.

Scaling a vector scales its length: $\|c\mathbf{x}\| = |c| \|\mathbf{x}\|$, so $\|2(3, 4)\| = \|(6, 8)\| = 10$. Dividing a vector by its own norm gives a *unit vector*, length exactly 1, pointing the same way: $(3, 4)/5 = (0.6, 0.8)$, and indeed $0.36 + 0.64 = 1$.

In NumPy: `np.linalg.norm(x)`; `np.dot(x, y)` or `x @ y` for the dot product.

3.3 The geometric meaning: the angle between two vectors

The dot product is also determined by the two lengths and the angle ϕ between the arrows:

$$\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos \phi, \quad \text{equivalently} \quad \cos \phi = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}.$$

This is the fact that makes the dot product useful. Read it as: *how much of $\mathbf{x}$ points along $\mathbf{y}$* , scaled by both lengths. It is large and positive when the two arrows point the same way (ϕ near 0, $\cos \phi$ near 1), zero when they are perpendicular ($\phi = 90^\circ$, $\cos \phi = 0$), and negative when they point in opposite directions (ϕ near 180° , $\cos \phi$ near -1).

Example. For $\mathbf{x} = (3, 4)$ and $\mathbf{y} = (4, 3)$,

$$\cos \phi = \frac{24}{5 \cdot 5} = 0.96, \quad \phi = \arccos(0.96) \approx 16.3^\circ.$$

For $\mathbf{x} = (3, 4)$ and $\mathbf{y} = (4, -3)$ the dot product was 0, so $\cos \phi = 0$ and the arrows are perpendicular; a sketch confirms it. For $\mathbf{x} = (3, 4)$ and $2\mathbf{x} = (6, 8)$, $\cos \phi = 50/(5 \cdot 10) = 1$: same direction, angle zero.

4 Euclidean distance

The *Euclidean distance* between two points is the length of the difference vector, the straight-line distance between them:

$$d_{\text{euc}}(\mathbf{x}^{(i)}, \mathbf{x}^{(k)}) = \|\mathbf{x}^{(i)} - \mathbf{x}^{(k)}\| = \sqrt{\sum_j (x_j^{(i)} - x_j^{(k)})^2}.$$

Example. With $\mathbf{x}^{(1)} = (3, 4)$, $\mathbf{x}^{(2)} = (4, 3)$, $\mathbf{x}^{(3)} = (0, 1)$:

$$d_{\text{euc}}(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}) = \|(-1, 1)\| = \sqrt{2} \approx 1.414,$$

$$d_{\text{euc}}(\mathbf{x}^{(1)}, \mathbf{x}^{(3)}) = \|(3, 3)\| = \sqrt{18} \approx 4.243,$$

$$d_{\text{euc}}(\mathbf{x}^{(2)}, \mathbf{x}^{(3)}) = \|(4, 2)\| = \sqrt{20} \approx 4.472.$$

So images 1 and 2 are close, and image 3 is far from both. This is the distance you would measure with a ruler on a drawing of the three points.

Three properties, all visible in the example: the distance from a point to itself is 0; $d_{\text{euc}}(\mathbf{x}, \mathbf{y}) = d_{\text{euc}}(\mathbf{y}, \mathbf{x})$; and a distance is never negative. They matter in §7.

Euclidean distance is sensitive to overall magnitude. A neuron population that fires twice as hard to every image moves every point twice as far from the origin and doubles every distance, even though the *pattern* across neurons is unchanged. The next two measures are designed to ignore that.

In NumPy: `np.linalg.norm(x - y)`.

5 Cosine similarity and cosine dissimilarity

Cosine similarity is just $\cos \phi$ from §3.3: the dot product with both lengths divided out. It compares directions and ignores lengths entirely. The course turns it into a dissimilarity (small means similar) by subtracting from one:

$$d_{\text{cos}}(\mathbf{x}^{(i)}, \mathbf{x}^{(k)}) = 1 - \cos \phi = 1 - \frac{\mathbf{x}^{(i)} \cdot \mathbf{x}^{(k)}}{\|\mathbf{x}^{(i)}\| \|\mathbf{x}^{(k)}\|}.$$

It is 0 when the two vectors point the same way, 1 when they are perpendicular, and 2 when they point in opposite directions; so it lives in $[0, 2]$.

Example. $\mathbf{x}^{(1)} = (3, 4)$ and $\mathbf{x}^{(2)} = (4, 3)$: $\cos \phi = 0.96$, so $d_{\text{cos}} = 0.04$. Nearly the same direction; nearly zero dissimilarity.

Example, where the two measures disagree. Take $\mathbf{x} = (1, 0)$ and $\mathbf{y} = (3, 0)$. Euclidean distance: $\|(-2, 0)\| = 2$. Cosine: $\cos \phi = 3/(1 \cdot 3) = 1$, so $d_{\text{cos}} = 0$. Euclidean says “two units apart”; cosine says “identical”. Neither is wrong: one asks *how far apart are the points*, the other *do the arrows point the same way*. If $\mathbf{y}$ is the same neurons firing three times as hard, cosine is the measure that says so.

In NumPy: `1 - np.dot(x, y) / (np.linalg.norm(x) * np.linalg.norm(y))`, for one pair of vectors.

6 Correlation and correlation dissimilarity

6.1 Centring a vector by its own mean

The *mean* of a vector is the average of its entries, $\bar{x} = \frac{1}{D} \sum_j x_j$. *Centring* subtracts it from every entry, so the centred vector $\tilde{x} = x - \bar{x}$ has mean zero. This is a per-vector operation: each stimulus is centred by its *own* mean, across its D entries.

Example. $x = (1, 2, 3)$ has mean 2, so $\tilde{x} = (-1, 0, 1)$. $y = (2, 4, 6)$ has mean 4, so $\tilde{y} = (-2, 0, 2)$. $z = (3, 2, 1)$ has mean 2, so $\tilde{z} = (1, 0, -1)$.

6.2 Pearson correlation is cosine similarity after centring

The *Pearson correlation* r between two vectors is the cosine of the angle between their centred versions:

$$r(x, y) = \frac{\tilde{x} \cdot \tilde{y}}{\|\tilde{x}\| \|\tilde{y}\|} = \frac{\sum_j (x_j - \bar{x})(y_j - \bar{y})}{\sqrt{\sum_j (x_j - \bar{x})^2} \sqrt{\sum_j (y_j - \bar{y})^2}}.$$

The second form is the textbook definition; the first says what it is: cosine on centred vectors. r lies in $[-1, 1]$. It is 1 when one vector is a positive-slope straight-line function of the other, -1 for a negative slope, and 0 when there is no linear relation.

Example. $\tilde{x} = (-1, 0, 1)$ and $\tilde{y} = (-2, 0, 2)$: $r = (2 + 0 + 2)/(\sqrt{2} \cdot \sqrt{8}) = 4/4 = 1$. Indeed $y = 2x$. $\tilde{x} = (-1, 0, 1)$ and $\tilde{z} = (1, 0, -1)$: $r = (-1 + 0 - 1)/(\sqrt{2} \cdot \sqrt{2}) = -1$. The pattern in z is the reverse of the pattern in x .

The course's third dissimilarity is

$$d_{\text{corr}}(x^{(i)}, x^{(k)}) = 1 - r,$$

again in $[0, 2]$: 0 for a perfectly matching pattern, 1 for no linear relation, 2 for a perfectly reversed one.

Why centre? Cosine ignores how *strongly* a population fires overall. Correlation also ignores its *baseline*: adding the same constant to every neuron's rate leaves r unchanged, because centring removes it. Correlation therefore compares only the *pattern* of ups and downs across neurons.

Example, where cosine and correlation disagree. Take $x = (5, 5, 6)$ and $y = (6, 5, 5)$. Both sit far from the origin in almost the same direction: $x \cdot y = 30 + 25 + 30 = 85$, $\|x\| = \|y\| = \sqrt{86}$, so $\cos \phi = 85/86 \approx 0.988$ and $d_{\text{cos}} \approx 0.012$. Now centre. Both means are $16/3$, so $\tilde{x} = (-\frac{1}{3}, -\frac{1}{3}, \frac{2}{3})$ and $\tilde{y} = (\frac{2}{3}, -\frac{1}{3}, -\frac{1}{3})$. Then $\tilde{x} \cdot \tilde{y} = -\frac{2}{9} + \frac{1}{9} - \frac{2}{9} = -\frac{1}{3}$, each centred norm is $\sqrt{6/9}$, and

$$r = \frac{-1/3}{6/9} = -0.5, \quad d_{\text{corr}} = 1.5.$$

Cosine calls them nearly identical because both are dominated by a large shared baseline (every neuron firing at about 5). Correlation strips the baseline and sees what is left: x

peaks on neuron 3, y peaks on neuron 1, opposite patterns. When the vectors are ReLU activations, which are non-negative and often share a large common offset, this difference between the two measures is not a technicality.

In NumPy: `np.corrcoef(x, y)[0, 1]` gives r for one pair of vectors.

6.3 The three dissimilarities, side by side

For two response vectors $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(k)}$:

$$d_{\text{euc}} = \left\| \mathbf{x}^{(i)} - \mathbf{x}^{(k)} \right\|, \quad d_{\text{cos}} = 1 - \cos \phi, \quad d_{\text{corr}} = 1 - r.$$

measure	ignores	range	0 means	example, (3, 4) vs. (4, 3)
d_{euc}	nothing	$[0, \infty)$	same point	$\sqrt{2} \approx 1.414$
d_{cos}	overall magnitude	$[0, 2]$	same direction	0.04
d_{corr}	magnitude and baseline	$[0, 2]$	same pattern	2

The last cell is worth checking by hand: (3, 4) centres to $(-0.5, 0.5)$ and (4, 3) to $(0.5, -0.5)$, exactly opposite, so $r = -1$ and $d_{\text{corr}} = 2$. With only two entries a pattern can only go up or down, so $D = 2$ correlations are always ± 1 ; with $D = 4096$ the measure is far more graded.

7 Matrices

7.1 What a matrix is, and two ways to read one

A matrix is a rectangular grid of numbers with m rows and n columns, “ $m \times n$ ”. Its entry in row i , column j is written with two subscripts, row first: $(\mathbf{A})_{ij}$ or a_{ij} . Two readings recur in this course. **As data:** $\mathbf{X}$ from §1.3, one row per stimulus, one column per feature. **As a map:** a matrix $\mathbf{W}$ turns a vector into a new vector, $\mathbf{w} \mapsto \mathbf{W}\mathbf{w}$, which is what a layer of a neural network does.

7.2 The matrix–vector product is a stack of dot products

If $\mathbf{A}$ is $m \times n$ and $\mathbf{w}$ has n entries, the product $\mathbf{A}\mathbf{w}$ is a vector with m entries, and **its i -th entry is the dot product of row i of $\mathbf{A}$ with $\mathbf{w}$:**

$$(\mathbf{A}\mathbf{w})_i = \sum_{j=1}^n a_{ij} w_j = \mathbf{a}_i \cdot \mathbf{w}, \quad \text{where } \mathbf{a}_i \text{ is row } i.$$

Example. With the 3×2 response matrix from §1.3 and $\mathbf{w} = (1, -1)$:

$$\mathbf{X}\mathbf{w} = \begin{pmatrix} 3 & 4 \\ 4 & 3 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 3-4 \\ 4-3 \\ 0-1 \end{pmatrix} = \begin{pmatrix} -1 \\ 1 \\ -1 \end{pmatrix}.$$

Read it as “project every stimulus onto the direction $\mathbf{w}$ ”: one number per image. The inner dimensions must agree: a 3×2 matrix takes a vector with 2 entries and returns one with 3. A matrix times a matrix works the same way, one column at a time: $(m \times k)(k \times n) \rightarrow (m \times n)$.

In NumPy: `A @ w`; the shapes must read $(m, n) @ (n,) \rightarrow (m,)$.

7.3 The transpose

The *transpose* $\mathbf{A}^\top$ swaps rows and columns: $(\mathbf{A}^\top)_{ij} = a_{ji}$, and an $m \times n$ matrix becomes $n \times m$.

$$\mathbf{X} = \begin{pmatrix} 3 & 4 \\ 4 & 3 \\ 0 & 1 \end{pmatrix} \implies \mathbf{X}^\top = \begin{pmatrix} 3 & 4 & 0 \\ 4 & 3 & 1 \end{pmatrix}.$$

Transposing a data matrix turns “one row per stimulus” into “one column per stimulus”. One product is worth knowing by name: $\mathbf{X}\mathbf{X}^\top$ is $N \times N$ and its entry (i, k) is the dot product of row i with row k , that is, $\mathbf{x}^{(i)} \cdot \mathbf{x}^{(k)}$, every pairwise dot product at once.

$$\mathbf{X}\mathbf{X}^\top = \begin{pmatrix} 3 & 4 \\ 4 & 3 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 3 & 4 & 0 \\ 4 & 3 & 1 \end{pmatrix} = \begin{pmatrix} 25 & 24 & 4 \\ 24 & 25 & 3 \\ 4 & 3 & 1 \end{pmatrix}.$$

The diagonal holds the squared norms (25, 25, 1, matching §3.2) and the off-diagonal entries are the dot products computed in §3.1. A matrix that equals its own transpose, as this one does, is called *symmetric*.

In NumPy: `X.T`; `X @ X.T`.

8 The dissimilarity matrix

8.1 Definition

Choose one of the three measures and compute it for *every pair* of stimuli. The result is a square $N \times N$ matrix $\mathbf{D}$, the *dissimilarity matrix* (the lectures also call it the RDM, representational dissimilarity matrix):

$$d_{ik} = d(\mathbf{x}^{(i)}, \mathbf{x}^{(k)}), \quad i, k = 1, \dots, N.$$

Entry d_{ik} answers one question: *how dissimilar are stimuli i and k , under this measure, in this representation?* Row i of $\mathbf{D}$ is stimulus i 's dissimilarity to everything.

Example. Euclidean, on the three images of §4:

$$\mathbf{D} = \begin{pmatrix} 0 & \sqrt{2} & \sqrt{18} \\ \sqrt{2} & 0 & \sqrt{20} \\ \sqrt{18} & \sqrt{20} & 0 \end{pmatrix} \approx \begin{pmatrix} 0 & 1.414 & 4.243 \\ 1.414 & 0 & 4.472 \\ 4.243 & 4.472 & 0 \end{pmatrix}.$$

8.2 Three properties every dissimilarity matrix has

They follow from the three properties of a distance noted in §4, and they are the first thing to check when you have computed one:

- **Zero diagonal:** $d_{ii} = 0$. A stimulus is not dissimilar to itself.
- **Symmetric:** $d_{ik} = d_{ki}$, so $\mathbf{D} = \mathbf{D}^\top$. The dissimilarity of i to k is the dissimilarity of k to i .
- **Non-negative:** $d_{ik} \geq 0$.

The same three hold for $d_{\cos}$ and d_{corr} ; only the range of the off-diagonal entries changes.

8.3 Counting the pairs

Because of the zero diagonal and the symmetry, an $N \times N$ dissimilarity matrix holds only $N(N-1)/2$ distinct numbers, the entries above the diagonal (the *upper triangle*). For $N = 3$ that is 3 numbers; for $N = 120$ it is 7,140. When you compare two dissimilarity matrices, those are the numbers you want: using all N^2 entries counts every pair twice and adds N zeros from the diagonal.

In NumPy: `np.allclose(D, D.T); np.allclose(np.diag(D), 0); (D >= 0).all()`.

9 Ranks and Spearman correlation

Sometimes only the *ordering* of a set of numbers is trustworthy, not their exact values. Rating scales are like this: a “7” is above a “6” but not necessarily one unit above. The *rank* of an entry is its position when the entries are sorted from smallest to largest.

Example. $x = (10, 20, 25)$ has ranks $(1, 2, 3)$; $y = (1, 5, 4)$ has ranks $(1, 3, 2)$. Ties get the average of the positions they occupy: $(4, 4, 9)$ has ranks $(1.5, 1.5, 3)$.

The *Spearman correlation* ρ is the Pearson correlation of the two rank vectors. It measures whether two variables increase together, without assuming the relation is a straight line. With no ties there is a shortcut:

$$\rho = 1 - \frac{6 \sum_j \delta_j^2}{n(n^2 - 1)}, \quad \delta_j = \text{rank}(x_j) - \text{rank}(y_j).$$

Example. Ranks $(1, 2, 3)$ and $(1, 3, 2)$ give $\delta = (0, -1, 1)$, $\sum \delta_j^2 = 2$, and $\rho = 1 - 6 \cdot 2 / (3 \cdot 8) = 0.5$. Computing Pearson r directly on the two rank vectors gives the same 0.5: centre both to $(-1, 0, 1)$ and $(-1, 1, 0)$, then $r = (1 + 0 + 0) / (\sqrt{2} \cdot \sqrt{2}) = 0.5$.

Two dissimilarity matrices are compared by taking the $N(N-1)/2$ upper-triangle entries of each, in the same pair order, and computing ρ between the two lists: a high ρ says the pairs one representation calls far apart, the other does too, whatever the units.

In NumPy: `scipy.stats.spearmanr(a, b).statistic`.

Symbols used in this handout

$\mathbf{x}, x_j$	a vector; its j -th entry	$\ \mathbf{x}\ $	Euclidean norm, $\sqrt{\mathbf{x} \cdot \mathbf{x}}$
D, N	entries per vector; number of stimuli	$\phi, \cos \phi$	angle between vectors; cosine similarity
$\mathbf{x}^{(i)}, x_j^{(i)}$	response to stimulus i ; entry j	$\bar{x}, \tilde{x}$	mean of $\mathbf{x}$; $\mathbf{x}$ centred by its own mean
$\mathbf{X}$	$N \times D$ response matrix	r, ρ	Pearson; Spearman (Pearson on ranks)
$\mathbf{x} \cdot \mathbf{y}$	dot product, $\sum_j x_j y_j$	$d_{\text{euc}}, d_{\text{cos}}, d_{\text{corr}}$	$\ \mathbf{x} - \mathbf{y}\ , 1 - \cos \phi, 1 - r$
$\mathbf{A}^\top$	transpose, rows and columns swapped	$\mathbf{D}, d_{ik}$	$N \times N$ dissimilarity matrix; entry (i, k)